Development of a Communication-Efficient Edge–Cloud Monitoring System for Wildlife Damage Control in Rural and Mountainous Areas

Kota Sato
Graduate School of Science and Technology
Shinshu University
Nagano, Japan
25w6039c@shinshu-u.ac.jp

Shungo Ito
Graduate School of Science and Technology
Shinshu University
Nagano, Japan
25w6010e@shinshu-u.ac.jp

Shimpei Sato
Faculty of Engineering
Shinshu University
Nagano, Japan
satos@shinshu-u.ac.jp

Daisuke Matsuo
Organization for IT Initiatives
Shinshu University
Nagano, Japan
daisuke_matsuo@shinshu-u.ac.jp

Lin Shan
Organization for IT Initiatives
Shinshu University
Nagano, Japan
shanlin@shinshu-u.ac.jp

***Abstract*—Wildlife damage to agricultural crops is a serious issue in rural and mountainous areas of Japan, where continuous monitoring is required under limited communication infrastructure and cost constraints. This paper presents a communication-efficient edge–cloud monitoring system using multiple trail cameras for wildlife damage control. The proposed system consists of three layers: edge cameras, an edge server, and a cloud server. Multiple cameras are connected to the edge server through a local area network, allowing multiple cameras to share a single cellular connection. To reduce the number of images forwarded to the cloud, the system employs a two-stage animal detection process. A lightweight AI model, YOLOv8n, is deployed on the edge server to filter out empty shots caused by environmental factors such as wind and vegetation movement, while MegaDetector v5a performs cloud verification before alerts are issued. Edge filtering reduced the number of capture events selected for transmission by 81.2% during daytime and 50.7% at night. Recall relative to MegaDetector reference labels was 93.8% and 68.4%, respectively, indicating a need to improve nighttime detection.**

***Index Terms*—Wildlife Monitoring, Edge-Cloud Architecture, Edge AI, Object Detection, Internet of Things (IoT)**

I. Introduction

Wildlife damage is a severe problem in Japan, with crop damage reaching 18.8 billion yen in FY2024; thus, wildlife control is an urgent issue [1].

Effective countermeasures require information on when and where animals are present, together with timely alerts. Conventionally, wildlife monitoring for this purpose has relied on automatic trail cameras equipped with passive infrared (PIR) sensors [2]–[4]. These devices record images on SD cards for later manual collection. IoT-based monitoring systems with remote notifications and satellite-connected camera traps have been developed to provide timely information [5]–[7].

However, large-scale deployment of communication-based cameras faces cost and efficiency challenges. First, false triggers can produce ”empty shots” containing no animals [3]. This increases communication traffic and requires manual image sorting. Second, individual LTE contracts and expensive camera units inflate initial and operational costs. Furthermore, existing edge AI cameras consume significant power, requiring bulky solar panels or high-capacity batteries for off-grid operation [8]. These factors restrict installation locations and act as major barriers for farmers and local governments.

To address these challenges, approaches such as deep learning for accurate animal detection [9], [10], empty shot elimination [2], [3], [11], and edge-device power saving [8], [11], [12] have been explored. Building on these approaches, we investigate how to distribute image acquisition, filtering, and notification across a shared monitoring network.

We propose a three-layer system comprising capture-only edge cameras, a shared edge server for YOLOv8n filtering, and a cloud server for MegaDetector verification and notification. A local WLAN connects the cameras to a shared LTE uplink, avoiding individual cellular contracts. Intended targets include wild boars, sika deer, Japanese macaques, Japanese serows, bears, raccoon dogs, and foxes. The system detects animal presence rather than identifying these species individually. We evaluate transmission reduction, agreement with reference detections, power consumption, communication range, and notification processing time.

II. Related Work

Deep learning has been widely used to streamline wildlife monitoring. Tabak et al. [2] trained convolutional neural network (CNN) models using a large-scale camera trap image dataset, reporting approximately 98% species-classification accuracy on their U.S. evaluation images. MegaDetector [3] is a general-purpose object detection model pre-trained on

camera-trap images from diverse geographic regions. Fennell et al. [13] reported that MegaDetector can detect animals accurately while limiting missed detections. Hitchcock et al. proposed a semi-automated workflow combining a general-purpose detector, a species classifier, and manual verification [4]. Mulero-Pázmány et al. combined a global model with group-specific expert models, including re-evaluation of images initially classified as background [14].

Satellite-based alerts extend monitoring beyond terrestrial coverage [5]. Raspberry Pi systems support edge detection with YOLOv8n [6] and tracking-based suppression of duplicate transmissions [15]. Other applications include YOLO-based intrusion detection [16], [17], PTZ tracking [18], and PIR-triggered alerts [19]. However, the power requirements of inference-capable camera nodes remain a concern for off-grid deployments [8], [12].

Sato et al. separated public-network communication into a parent node to reduce the power requirements of child nodes, which retained animal detection and alerting functions [8]. To reduce system costs and communication delays, a hierarchical edge computing approach that delegates main processing to gateway devices like Raspberry Pi has been proposed [20]. Furthermore, an IoT architecture linking edge inference with a cloud database has also been constructed [7]. When inference is performed at each camera, computational and power requirements must be addressed at every deployed node. Saito et al. used a low-power auxiliary board to switch off the Raspberry Pi during idle periods [12]. More recently, SPARROW has connected peripheral cameras and sensors to a local hub for edge processing before cellular or satellite transmission [21]. This provides a relevant precedent for aggregating sensing data and computation before wide-area communication. Harits et al. implemented an aquaculture system linking ESP32 sensor nodes to a Raspberry Pi 4 gateway for intrusion prevention and encryption before cloud storage [22], illustrating an intermediate processing layer between sensors and cloud services. The present study applies local aggregation to wildlife damage monitoring, with capture-only cameras, lightweight filtering on a shared edge server, and cloud-side animal verification.

Monitoring can also support active countermeasures, including remotely activated monkey traps [23] and species-dependent ultrasonic deterrents [24]. Integration with such systems is a possible future application of the monitoring network.

III. Proposed System

The proposed system separates image acquisition, edge filtering, and cloud verification. Cameras transfer images without performing inference, with battery-powered operation as a design goal. Following the communication aggregation concept in [8] and the local-hub approach also used by SPARROW [21], a shared edge server filters capture events before wide-area transmission; the cloud provides a second check before notification.

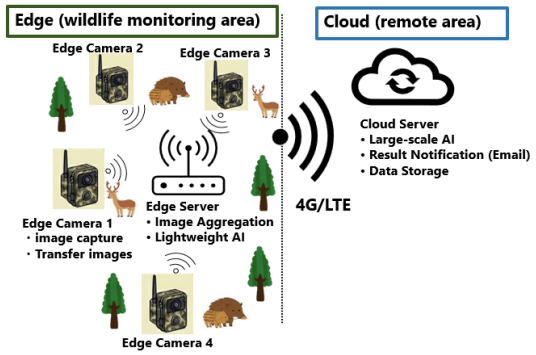


Fig. 1. Overall architecture of the proposed system.

A. System Architecture

The overall configuration of the proposed system is shown in Fig. 1. This system consists of the following three main nodes and the network connecting them:

1) **Edge Camera**: Image acquisition and transfer to the edge server.
2) **Edge Server**: Image aggregation, edge filtering with YOLOv8n, and transfer to the cloud server.
3) **Cloud Server**: Cloud verification with MegaDetector v5a, data storage, and user notification.
4) **Network Configuration**: Local camera connections and an LTE uplink to the cloud.

1) Edge Camera: Edge cameras are deployed directly throughout the monitoring area. They capture images when triggered by a PIR sensor and transfer them to the edge server via local communication. The PIR sensor provides a motion trigger; animal presence is determined by AI models at the edge and cloud servers. Battery-powered operation in locations without commercial power is a design goal.

The prototype uses the Seeed Studio XIAO ESP32S3 Sense. This device provides Wi-Fi communication and camera functions. Furthermore, its Deep Sleep function is used with the aim of reducing standby consumption; the measured power limitations of the prototype are discussed in Section V. A PIR sensor triggers image capture in response to motion, and HTTP communication via Wi-Fi is used to transmit images to the edge server. It uses eight AA batteries (6V) as a power source, facilitating installation in locations without power infrastructure. Fig. 2 shows an overview of the edge camera prototype.

2) Edge Server: The edge server connects multiple cameras to a shared LTE uplink. It performs edge filtering with YOLOv8n to select capture events with animal detections for cloud transfer. To perform these processes autonomously in outdoor environments, the small computer and communication module must be capable of continuous operation using solar panels or similar power sources.

In our implementation, a Raspberry Pi 4B is used as the small computer, and an NEC Aterm HT100LN is used as the communication module. Edge filtering uses YOLOv8n pre-

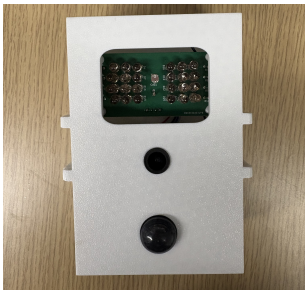


Fig. 2. Prototype of the edge camera.

trained on the COCO dataset. Its detection performance and processing time are evaluated in Section IV. HTTP communication is also used for transferring images to the cloud server. To operate this configuration continuously in an environment without commercial power, an approximately 200W solar panel and a 100Ah battery are considered as a preliminary power configuration. Final sizing requires the battery voltage, usable capacity, conversion losses, workload, and site-specific solar availability to be specified; autonomous operation with this configuration has not been validated.

3) Cloud Server: The cloud server verifies images forwarded by the edge server using MegaDetector v5a. This provides a second check for animal presence before notification, with the aim of reducing false alerts. Received images and detection results are stored for future analysis or model retraining.

MegaDetector is pre-trained on camera-trap images from diverse environments and is used here for animal-presence verification rather than species identification.

4) Network Configuration: The network has two stages. First, cameras transfer images through a local WLAN to the shared edge server, eliminating the need for individual cellular contracts. Second, the edge server forwards selected events to the cloud via HTTP over LTE. The NEC Aterm HT100LN provides both the 2.4GHz Wi-Fi access point for local connectivity and the LTE uplink. This arrangement shares one wide-area connection among cameras; coverage and concurrent operation require deployment-specific evaluation.

B. Data Processing Flow

The data processing flow of the proposed system is shown in Fig. 3. Processing proceeds from image acquisition at the camera to edge filtering at the edge server, followed by cloud verification and notification. The processing flow of each component is described below.

1) Edge Camera Processing Flow: The edge camera maintains a standby state using Deep Sleep mode and wakes when triggered by the PIR sensor. After waking up, it captures three consecutive images in burst mode and immediately transmits them to the edge server via HTTP communication. After transmission is complete, the edge camera returns to the standby state after a certain interval, to reduce energy consumption.

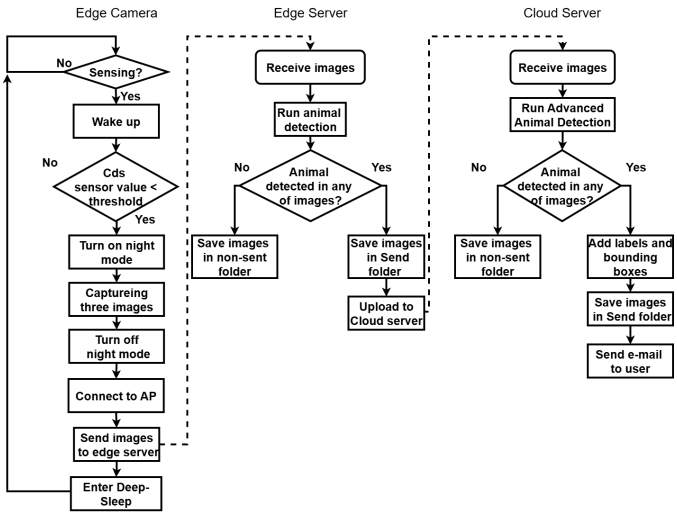


Fig. 3. Data processing flow of the proposed system.

2) Edge Server Processing Flow: Upon receiving images from a camera, the edge server runs YOLOv8n for edge filtering. If a class corresponding to an animal is detected in at least one of the three received images, the images are moved to a storage directory and transferred to the cloud server. If no animal is detected (e.g., empty shots due to swaying vegetation), the images are not transmitted, reducing the frequency of image transmissions.

3) Cloud Server Processing Flow: Upon receiving images from the edge server, the cloud server performs verification using MegaDetector v5a. The server sends an email alert with the image only when an animal is detected. All received images and detection results are stored in the cloud, regardless of whether an alert is issued.

IV. Experimental Results

In this section, we describe the following four experiments conducted to evaluate the prototype:

1) Measurement of power consumption during operation of each node.
2) Evaluation of communication performance in an outdoor line-of-sight environment.
3) Evaluation of detection agreement with MegaDetector using a dataset collected in Kiso Town, Nagano Prefecture.
4) Measurement of system processing time.

A. Experimental Environment and Parameters

The equipment configuration used for the prototype experiments is shown in Table I. The detection evaluation separately uses previously collected images from commercial cameras in Kiso Town, rather than images captured by the prototype edge camera. Communication tests between the edge camera and the edge server were conducted in a line-of-sight outdoor environment on a sunny day.

TABLE I
SYSTEM COMPONENTS

Component	Model	Remarks
Edge Camera		
Main Unit	Seeed Studio XIAO ESP32S3 Sense	Microcontroller with Wi-Fi/Camera
PIR Sensor	Panasonic PaPIRs	Detection range ∼14m
Power Supply	8 × AA batteries	6V
Edge Server		
Main Unit	Raspberry Pi 4B	4GB RAM
Power Supply	Preliminary (Est.)	∼200W panel, 100Ah battery
Comms Module	NEC Aterm HT100LN	LTE router

TABLE II
POWER CONSUMPTION OF EDGE CAMERA (5.0V)

Operating State	Current (mA)	Power (W)
Standby	114	0.57
Startup (Peak)	946	4.73
Wi-Fi Transmission	250	1.25

B. Power Consumption Evaluation of Edge Camera

Because this system assumes operation in agricultural and mountainous areas where commercial power is difficult to access, the power consumption of each node is a critical indicator determining practicality. We measured the power consumption of the edge camera in each operating state (Table II). At a power supply voltage of 5.0V, the standby current of the entire edge camera node was 114mA, and the peak startup current was 946mA.

C. Power Consumption Evaluation of Edge Server

We similarly measured the power consumption of the edge server (Raspberry Pi 4B). At 5.0V, the idle current was 405∼410mA (approx. 2.05W). Adding the maximum power consumption of the LTE router (approx. 6W), the resulting power estimate is approximately 8W. This combines the measured Pi idle consumption with the router maximum rating, rather than measuring the combined system under inference load.

D. WLAN Communication Performance

Under line-of-sight conditions, we evaluated communication stability while varying the distance between the edge camera and the server. Table III shows the experimental results. At close range (approx. 1m), RSSI remained stable at around -30∼-35dBm. At a distance of 100m, RSSI was around -60∼-65dBm, and it dropped to -77∼-80dBm at the maximum distance of 200m. While connection establishment and image transfer were successfully completed without problems up to 100m, approaching the 200m point caused RSSI to fall below -75dBm, resulting in noticeable latency and communication instability.

TABLE III
COMMUNICATION PERFORMANCE (LINE-OF-SIGHT ENVIRONMENT)

Distance (m)	RSSI (dBm)
≈ 1	-30 ∼ -35
≈ 100	-60 ∼ -65
≈ 200	-77 ∼ -80

TABLE IV
EVENT-LEVEL AGREEMENT WITH MEGADETECTOR (YOLO: 0.05; MD: 0.25)

		Daytime (YOLOv8n)		Nighttime (YOLOv8n)	
		Pos.	**Neg.**	**Pos.**	**Neg.**
Ref (MD)	**Pos.**	**195**	13	**104**	48
	Neg.	73	**1,147**	28	**88**
Precision		72.8%		78.8%	
Recall		93.8%		68.4%	

E. Reduction in Capture Events Selected for Transmission

We evaluated YOLOv8n using images collected by commercial cameras in Kiso Town from June to November 2025. The dataset contains the intended wildlife targets listed in Section I. The evaluation unit is a capture event, normally comprising three images: there are 1,428 daytime events (4,284 images) and 268 nighttime events (804 images). A model labels an event positive if it detects an animal in at least one available image. YOLOv8n uses a confidence threshold of 0.05 to limit missed detections, while MegaDetector v5a uses 0.25 to generate reference labels (Table IV). These labels are model-generated rather than human-annotated ground truth. Precision is the fraction of YOLO-positive events that are also MD-positive; recall is the fraction of MD-positive events that are YOLO-positive. Thus, the reported metrics describe agreement with MegaDetector, not independently verified animal detection accuracy.

Relative to the MegaDetector reference, daytime recall and precision were 93.8% and 72.8%, respectively; nighttime recall and precision were 68.4% and 78.8%. Fig. 4 shows a daytime detection and a nighttime missed detection from the evaluation images.

Next, Table V shows the transmission reduction rate on a capture-event basis. This is the fraction of events not selected for forwarding because YOLOv8n detects no animal. It includes events that are positive according to MegaDetector but negative according to YOLOv8n. In Equation 1, N_{total} is the total number of capture events, N_{selected} is the number selected for transmission, and R is the reduction rate in percent. The equation counts events rather than individual images or transmitted bytes.

$$R = \left(1 - \frac{N_{\mathrm{selected}}}{N_{\mathrm{total}}}\right) \times 100 \tag{1}$$

The event-based transmission reduction rates were 81.2% during daytime and 50.7% at night.



(a) Daytime: animal detected.



(b) Nighttime: animal missed.

Fig. 4. Example images from the Kiso Town evaluation dataset, captured by commercial cameras. At a confidence threshold of 0.05, YOLOv8n detects the animal in (a) but misses the animal in the lower part of (b). These are image-level examples.

TABLE V
TRANSMISSION REDUCTION BY DAY/NIGHT (EVENT BASIS)

Item	Daytime	Nighttime
Total Events	1,428	268
Events Selected for Transmission	268	132
Reduction Rate (%)	81.2	50.7

F. Total System Processing Time

We measured system processing time from PIR-triggered motion sensing to completion of email notification processing in a stable communication environment with an RSSI of around -35dBm. The edge-side processing time, from the PIR trigger to completion of transmission processing at the edge server, was approximately 33 seconds: approximately 20 seconds for camera processing and 13 seconds for edge server processing. Cloud-side processing, from image reception to completion of email notification processing, required approximately 11 seconds. The total processing time from the PIR trigger was approximately 44 seconds.

V. DISCUSSION

In the experiment, event-based transmission reductions of 81.2% during the daytime and 50.7% at night were achieved. These reductions include missed detections: YOLOv8n rejected 13 of 208 MD-positive daytime events and 48 of 152 MD-positive nighttime events. Cloud verification cannot recover events discarded at the edge. Nighttime recall was 68.4%. In Fig. 4(b), the animal is partly obscured by vegetation and has limited contrast with its surroundings. These conditions and the limitations of the lightweight model may contribute to missed detections, but their individual effects have not been isolated. Transfer learning using nighttime camera images and comparison with alternative lightweight models should evaluate both recall and transmission reduction. Independent human annotations are needed to establish detection accuracy beyond agreement with MegaDetector.

Power consumption also limits deployment. At the measured whole-node standby power of 0.57W, 35 hours of standby alone requires approximately 20Wh delivered to the camera. This is an energy requirement, not a measured battery lifetime; capture, transmission, conversion losses, and usable battery capacity must also be considered. Potential improvements include switching off peripherals such as the SD card and camera when inactive and shortening active capture and communication periods. Component-level measurements are needed to identify the sources of standby consumption. SPARROW uses scheduled uplink operation and event-triggered communication [21]; a similar duty-cycling strategy warrants evaluation here alongside its effect on notification latency.

The current results support considering deployments where cameras have clear WLAN paths of approximately 100m to an edge server with reliable power and LTE coverage, and where approximately 44 seconds from the PIR trigger to completion of email notification processing is acceptable. The sunny, line-of-sight communication test does not establish performance in dense vegetation, obstructed terrain, or adverse weather. Communication became unstable near 200m; repeaters or alternative links should be evaluated for longer distances. Nighttime recall and battery endurance require improvement before unattended long-term use. Long-term field operation and concurrent multi-camera performance remain to be evaluated.

VI. CONCLUSION

This paper presents an edge–cloud monitoring system for wildlife damage control. It uses a three-layer structure: edge cameras, an edge server, and a cloud server. This separates image acquisition, filtering, and notification across the three layers. The architecture connects multiple cameras to a shared LTE uplink through a local WLAN, eliminating the need for

individual cellular contracts. The prototype combines edge filtering with YOLOv8n and cloud verification with MegaDetector v5a. We confirmed event-based transmission reductions of 81.2% during the daytime and 50.7% at night, with recall relative to MegaDetector of 93.8% and 68.4%, respectively. Nighttime detection and power consumption remain key limitations.

Future work will prioritize human-annotated evaluation, nighttime transfer learning, and peripheral power control. We will measure complete capture–transfer–sleep cycles and battery endurance, then evaluate obstructed links, concurrent cameras, and long-term field operation. Model and communication changes will be assessed against recall, transmission reduction, and notification latency.

ACKNOWLEDGMENT

This research and development work was supported by the Ministry of Internal Affairs and Communications (MIC), Japan, awarded through the FORWARD program under grant #JPMI250420001 (Principal Investigator: Lin Shan).

REFERENCES

[1] Ministry of Agriculture, Forestry and Fisheries, "Damage to crops by wildlife in japan," Available: https://www.maff.go.jp/j/seisan/tyozyu/higai/hogai_zyoukyou/index.html (in Japanese), 2026, (Accessed Feb. 10, 2026).

[2] M. A. Tabak, M. S. Norouzzadeh, D. W. Wolfson, S. J. Sweeney, K. C. VerCauteren, N. P. Snow, J. M. Halseth, P. A. Di Salvo, J. S. Lewis, M. D. White, B. Teton, J. C. Beasley, P. E. Schlichting, R. K. Boughton, B. Wight, E. S. Newkirk, J. S. Ivan, E. A. Odell, R. K. Brook, P. M. Lukacs, A. K. Moeller, E. G. Mandeville, J. Clune, and R. S. Miller, "Machine learning to classify animal species in camera trap images: Applications in ecology," *Methods in Ecology and Evolution*, vol. 10, no. 4, pp. 585–590, April 2019.

[3] S. Beery, D. Morris, and S. Yang, "Efficient pipeline for camera trap image review," arXiv preprint arXiv:1907.06772, July 2019.

[4] K. Hitchcock, S. Tollington, R. W. Yarnell, L. J. Williams, K. Hamill, and P. Fergus, "Optimizing the accuracy and efficiency of camera trap image analysis: Evaluating AI model performance and a semi-automated workflow," *Remote Sensing*, vol. 18, no. 3, p. 502, 2026. [Online]. Available: https://www.mdpi.com/2072-4292/18/3/502

[5] R. C. Whytock, T. Suijten, T. van Deursen, J. Świeżewski, H. Mermiaghe, N. Madamba, N. Mouckoumou, J. A. Zwerts, A. F. K. Pambo, L. Bahaa-el din, S. Brittain, A. W. Cardoso, P. Henschel, D. Lehmann, B. R. Momboua, L. Makaga, C. Orbell, L. J. T. White, D. M. Iponga, and K. A. Abernethy, "Real-time alerts from AI-enabled camera traps using the Iridium satellite network: A case-study in Gabon, Central Africa," *Methods in Ecology and Evolution*, vol. 14, no. 3, pp. 867–874, 2023.

[6] A. Patkar, P. Hole, and L. Salvi, "Real-time wildlife intrusion detection system using IoT and YOLOv8," *Journal of Smart Sensors and Computing*, vol. 1, no. 2, p. 25209, September 2025.

[7] I. A. Zualkernan, S. Dhou, J. Judas, A. R. Sajun, B. R. Gomez, L. A. Hussain, and D. Sakhnini, "Towards an IoT-based deep learning architecture for camera trap image classification," in *2020 IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, 2020, pp. 1–6.

[8] R. Sato, H. Saito, Y. Tomioka, and Y. Kohira, "Energy reduction methods for wild animal detection devices," *IEEE Access*, vol. 10, pp. 24 149–24 161, 2022.

[9] A. D. Shetty and S. Ashwath, "Animal detection and classification in image & video frames using YOLOv5 and YOLOv8," in *2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, 2023, pp. 677–683.

[10] C. S. Madhumathi, V. Naveen, N. Akshay, M. S. Kumar, and M. M. Aslam, "Advanced wild animal detection and alert system using YOLO V5 model," in *2023 7th International Conference on Trends in Electronics and Informatics (ICOEI)*, 2023, pp. 365–371.

[11] D. Velasco-Montero, J. Fernández-Berni, R. Carmona-Galán, A. Sanglas, and F. Palomares, "Reliable and efficient integration of AI into camera traps for smart wildlife monitoring based on continual learning," *Ecological Informatics*, vol. 83, p. 102815, 09 2024.

[12] H. Saito, T. Otake, H. Kato, M. Tokutake, S. Semba, Y. Tomioka, and Y. Kohira, "Battery-powered wild animal detection nodes with deep learning," *IEICE Transactions on Communications*, vol. E103.B, no. 12, pp. 1394–1402, Dec. 2020.

[13] M. Fennell, C. Beirne, and A. C. Burton, "Use of object detection in camera trap image identification: Assessing a method to rapidly and accurately classify human and animal detections for research and application in recreation ecology," *Global Ecology and Conservation*, vol. 35, p. e02104, 03 2022.

[14] M. Mulero-Pázmány, S. Hurtado, C. Barba-González, M. L. Antequera-Gómez, F. Díaz-Ruiz, R. Real, I. Navas-Delgado, and J. F. Aldana-Montes, "Addressing significant challenges for animal detection in camera trap images: a novel deep learning-based approach," *Scientific Reports*, vol. 15, no. 1, p. 16191, 2025.

[15] M. A. Rahman, S. Giordano, and M. Pagano, "Smart wildlife monitoring: Real-time hybrid tracking using Kalman filter and local binary similarity matching on edge network," *Computers*, vol. 14, p. 307, 07 2025.

[16] K. M, B. V, and A. K, "Animal intrusion detection using Yolo V8," in *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*, vol. 1, 2024, pp. 206–211.

[17] K. Selvaraj, M.Anandaraj, A. J, and S.Saravanan, "An efficient automated framework for wildlife detection and prevention in agricultural zones nearer to forest reserves," in *2025 6th International Conference on Smart Electronics and Communication (ICOSEC)*, 2025, pp. 1659–1664.

[18] S. Semba, H. Saito, Y. Tomioka, and Y. Kohira, "A battery-powered wild animal tracking device using a PTZ camera and deep learning," in *Proc. 2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 10 2024, pp. 3181–3186.

[19] S. K S, S. S, S. B. K, S. Doss, J. L, and S. K. G, "Eyes in the thicket: A neural gaze for invisible wildlife encounters," in *2025 Control Instrumentation System Conference (CISCON)*, 2025, pp. 1–6.

[20] N. Monburinon, S. M. S. Zabir, N. Vechprasit, S. Utsumi, and N. Shiratori, "A novel hierarchical edge computing solution based on deep learning for distributed image recognition in IoT systems," in *2019 4th International Conference on Information Technology (InCIT)*, 2019, pp. 294–299.

[21] J. M. Lavista Ferres, C. Chalmers, B. Demuro Segundo, Z. Miao, A. Hernandez Celis, F. Alves Torres, I. D. Chacon Silva, A. Cintron Roman, A. Kim, M. Machado, L. Marotti, A. Michaels, D. Ruiz Lopez, C. Romero, R. Dodhia, I. Becker-Reshef, and P. Arbelaez, "Project SPARROW and the future of conservation technology," arXiv preprint arXiv:2606.00108v1, 2026. [Online]. Available: https://arxiv.org/abs/2606.00108v1

[22] M. A. Harits, M. R. Aditria, T. A. Fathi, N. Karna, and Istikmal, "Edge computing IoT security (ECIS): Implementation of IPS and encryption for my I-Pond," in *2025 8th International Conference on Information and Communications Technology (ICOIACT)*, 2025, pp. 24–29. [Online]. Available: https://doi.org/10.1109/ICOIACT67584.2025.11345049

[23] R. Kamesaka and Y. Hoshino, "Development of a prevention system for beast damage of agricultural products using deep learning," in *2018 Joint 10th International Conference on Soft Computing and Intelligent Systems (SCIS) and 19th International Symposium on Advanced Intelligent Systems (ISIS)*, 2018, pp. 747–752.

[24] V. K. Manavalasundaram, M. B, Y. K. L, P. T, and V. S, "Smart wildlife detection and repulsion system using deep-learning," in *2025 4th International Conference on Automation, Computing and Renewable Systems (ICACRS)*, 2025, pp. 1–8.